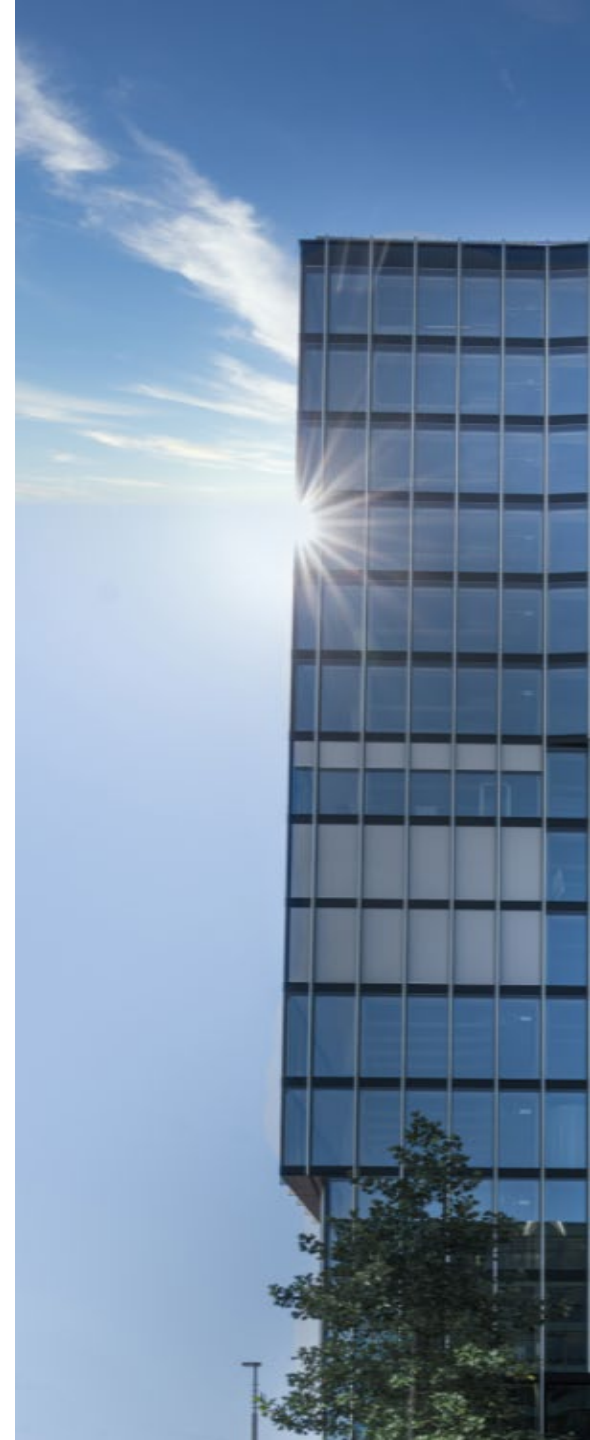


Objektorientierte Programmierung

Stream API - Datenströme

Roland Gisler



Inhalt

- Konzept der Datenströme
- Beispiel: Klassisch Imperativ vs. Deklarativ mit Stream API
- Stream Processing in Java
- Grundlagen des Stream Processing
- Beispiele
- Exkurs: Auto[Un]Boxing
- Wichtige Hinweise und Empfehlungen
- Zusammenfassung

Lernziele

- Sie kennen das Prinzip des «Stream»-Programming.
- Sie können einfache Beispiele die auf Streams und funktionaler Programmierung (Lambda) basieren nachvollziehen.
- Sie kennen das Konzept der Supplier-, Intermediate- und Terminal-Operationen für/auf Streams.
- Sie können einfache Anforderungen mittels der Stream API umsetzen.
- Sie wissen was AutoBoxing ist, können damit adäquat umgehen und sind sich der Konsequenzen bewusst.

Konzept der Datenströme

Was ist ein Datenstrom?

0 1 0 1 1 0 0 1 1 0 0 0 1 0 1 0 1 0 1 0 1 0 0 1 1 0 1 1 1

Die typischen Merkmale eines Datenstromes (Stream) sind:

- Kontinuierlicher Fluss von Datenelementen.
 - Ende (der Menge) meist nicht von Anfang an bekannt.
 - Können in kleinen Einheiten und laufend verarbeitet werden.
 - Datenrate (Menge/Zeit) kann variieren (je nach Quelle).
 - Kein wahlfreier Zugriff (vgl. Datenstrukturen, z.B. **ArrayList**)
- Bekannte Beispiele: Audio- oder Video-Stream (auch Live).
 - Das Konzept ist universell nutzbar: Datenströme müssen nicht nur binär sein, sondern können auch von elementaren Datentypen oder beliebigen Objekten (von Klassen) sein.
 - z.B. ein Strom von **Temperatur**- oder **Person**-Objekten!

Vorteile durch die Stream-Verarbeitung

- Daten können unmittelbar und fortlaufend verarbeitet werden.
 - Man muss nicht warten bis «alles» da ist.
 - Somit keine vollständige Speicherung (im Memory) notwendig.
- Man kann somit auch mit sehr grossen Datenmengen einfach umgehen.
- Die Verarbeitung kann unter Umständen sehr einfach (deklarativ) parallelisiert werden.
Analogie: Mehrere Kassen in einem Supermarkt, die nach bedarf besetzt werden.
 - Fork/Join-Prinzip → Thema Nebenläufigkeit im Modul AD.
- Der Programmcode wird bzw. kann dadurch einfacher werden.
 - Leichter verständliche, deklarative Lösungskonzepte möglich.
- Verbindung mit funktionaler Programmierung (in Java mit Lambdas) vereinfacht das Programmiermodell zusätzlich.

Abgrenzung: I/O-Streams versus Stream API

- **Wichtig:** I/O-Streams und Stream API sind **verschiedene** Dinge!
- I/O-Streams:
 - Package `java.[n]io`, I/O-Verarbeitung, teilweise seit Java 1.0.
 - Lesen und schreiben von Daten (Bytes und Zeichen) aus I/O-Quellen und -Senken wie z.B. Dateien, Netzwerk, Konsole etc. auf sehr technischer Ebene.
- Stream API:
 - Package `java.util.stream`, Datenströme, erst seit Java 8.0.
 - Verarbeitung (primär) von Objektströmen.
 - Typische Operationen: Suchen, Filtern, Mapping etc.
 - Die Stream API arbeitet auf einer abstrakteren Ebene und ist eng mit der funktionalen Programmierung mit Hilfe von → Lambdas verknüpft.

Ein konkretes Beispiel

Datenverarbeitung – Imperative Programmierung

- Gegeben sei eine Liste `verlauf` mit Temperatur-Objekten.
 - Enthaltene Temperaturwerte: `22.1` / `20.2` / `25.3` / `23.7` / `19.9`
- Welche Anforderung setzt das folgende Codefragment um?

```
final List<Temperatur> selected = new ArrayList<>();
for (final Temperatur t : verlauf) {
    if (t.getCelsius() >= 20f && t.getCelsius() <= 25f) {
        selected.add(t);
    }
}
Collections.sort(selected);
for (final Temperatur t : selected) {
    LOG.info("{}°C", t.getCelsius());
}
```

```
2023-05-12 22:11:22,011 INFO - 20.2°C
2023-05-12 22:11:22,518 INFO - 22.1°C
2023-05-12 22:11:22,724 INFO - 23.7°C
```

Beispiel – Klassische, imperative Datenverarbeitung

- Die Aufgabe des Codebeispiels von oben war:
 - Aus der Liste **verlauf** mit **Temperatur**-Objekten sollen...
 - alle Temperaturen zwischen **20°** bis **25°** Celsius herausgefiltert,
 - dann aufsteigend nach Wert sortiert,
 - und dann einzeln in der Einheit °Celsius ausgegeben werden.
- Das ist eine **völlig typische** Verarbeitung, wie sie in dieser Form in verschiedensten Domänen immer und immer wieder vorkommt, und bis Java 7 nicht anders möglich war!
 - ➔ Das ist ganz klassische und typische **imperative** Programmierung.
- «Überraschende» Erkenntnisse:
 - Bewusster Einsatz eines temporären Zwischenspeichers (**selected**) nötig.
 - Die Anforderungen waren nicht wirklich «so» einfach aus dem Code zu lesen.
 - Relativ viel (boilerplate-)Code, z.B. mit Hilfsvariablen, Iteratoren etc.

Lösung mit Stream-API als Pipeline

Beispiel - Lösung in funktionaler Programmierung

- Identische Aufgabe, Lösung mit Stream API und Lambdas:

```
verlauf.stream()  
    .map(t -> t.getCelsius())  
    .filter(c -> (c >= 20.0f && c <= 25.0f))  
    .sorted()  
    .forEach(v -> LOG.info("{}°C", v));
```

- Was sofort auffällt:
 - Es ist deutlich leserlicher - nach einer gewissen Gewöhnungszeit! 😊
 - Der Code ist wesentlich **kürzer** und **ausdruckstärker**, es ist tatsächlich ein **Einzeiler**, und liest sich (fast) wie natürliche Sprache!
- **Deklarative Programmierung**: Wir konzentrieren uns auf «**Was**» man machen will, und nicht auf das «**Wie**»: Weder die Iterationen noch die (impliziten) Zwischenspeicher sind im Code sichtbar! **Wow!**

Stream-Processing in Java

Stream API - Java Collection Framework

- Parallel zu den Lambdas wurde im Java Collection Framework die so genannte **Stream API** ergänzt.
- Die Stream API erlaubt es Daten z.B. von Collections (als eine der häufigsten Quellen) als Datenstrom zu verarbeiten: Mittels der **stream()**-Methode kann von jeder Collection ein «Strom» aller enthaltenen Elemente erzeugt werden!
- Das Geniale: Es ist möglich, diese Datenströme mittels → Pipelines (eine kaskadierte Folge von Methodenaufrufen) zu verarbeiten!
 - Typisch: Anwenden von Lambdas auf den Inhalt eines Streams.
- Beispiele:
 - Prädikat über **filter(...)**, Mapping über **map(...)**
 - Details siehe JavaDoc des [Packages `java.util.stream`](#).

Streamverarbeitung - Pipeline

- Diese Verkettung von Funktionen nennt man **Pipeline**.

```
verlauf.stream()  
    .map(t -> t.getCelsius())  
    .filter(c -> (c >= 20.0f && c <= 25.0f))  
    .sorted()  
    .forEach(v -> LOG.info("{}°C", v));
```

- Eine Pipeline setzt sich typisch aus
 - einem **Supplier** (Quelle) – z.B. `stream()`, `Stream.of(...)` etc.
 - 0..n **Intermediate**-Methode(n) – z.B. `filter(...)`, `map(...)` etc.
 - 0..1 **Terminal**-Methode – z.B. `forEach(...)`, `max(...)` etc.
- Intermediate-Methoden liefern wieder einen (veränderten) **Stream**.
 - z.B. veränderte Datenmenge und/oder Typen/Werte.
- Terminal-Methoden liefern entweder **ein** Ergebnis oder **void**.

Grundlagen des Stream Processing

Stream API - Supplier (Quellen) für Streams

- Von Java-Collections kann mittels `stream()` direkt ein Stream erzeugt werden.
Eine der häufigsten Anwendungen als Beispiel:

```
final List<Temperatur> verlauf = verlauf.stream().forEach(...);
```

- Man kann Streams aber auch Deklarativ erzeugen:

```
Stream.of("Hedwig", "Hagrid", "Hermine").forEach(...);
```

- Oder Streams von/für elementare Datentypen erzeugen:

```
IntStream.range(0, 10).forEach(...);
```

- Die letzte Variante ermöglicht ganz neue Lösungskonzepte für iterative Aufgabenstellungen!
 - z.B. als Ersatz für `for(...)`-Iterationskonstrukte (siehe O05).

Stream API - Intermediate Operationen

- Verschiedene Intermediate-Operationen «verarbeiten» jedes Element eines Streams und liefern wiederum einen Stream als Resultat.
 - Die Verarbeitung kann z.B. eine Filterung mit `filter(...)` oder eine Transformation mit `map(...)` sein.

- Beispiel für Filterung: Man wendet die Methode

```
Stream<T> filter(Predicate<? super T> predicate);
```

an, welche als Parameter ein Lambda für die Methode `boolean test(T t)` entgegen nimmt.

- Abhängig vom bool'schen Resultat werden die gewünschten Elemente aus dem Quell-Stream selektiert (`true`). Beispiel:

```
verlauf.stream().filter(t -> t.isHot()).forEach(...);
```

Stream API - Terminal-Operationen

- Terminal-Operationen **konsumieren** und schliessen einen Stream ab.
 - Ein konsumierter Stream kann nicht weiter verwendet werden.

Typische Anwendungen sind:

- Auf jedem Element eine Methode aufrufen. Beispiel:

```
verlauf.stream().forEach(System.out::println);
```

- Elemente aggregieren. Beispiel:

```
verlauf.stream().min();
```

- Alle Elemente in eine Collection (hier Set) übertragen. Beispiel:

```
verlauf.stream().collect(Collectors.toSet());
```

Beispiele

Beispiel 1 – Eine Anforderung, verschiedene Lösungen

- Einfache Aufgabe: Geben Sie die Zahlen von **1** bis **10** aus.
- Klassische Lösung mit einer Iteration (**for**-Loop, SW04):

```
for(int i = 1; i < 11; i++) {           // oder <= 10
    System.out.println(i);
}
```

- Lösung über einen Stream (**IntStream**):

```
IntStream.range(1, 11).forEach(System.out::println);
```

- Zeigt sehr schön den Unterschied zwischen **imperativer** und **deklarativer** Lösung!
 - Beurteilen Sie selber: Welche Lösung finden Sie besser?
 - Robuster? Verständlicher? Wartungsfreundlicher?

Beispiel 2 – Einfache, effiziente Lösungen wählen!

- Aufgabe: Maximalen Wert aus einer Zahlenfolge ausgeben.
- **Gute**, einfache Lösung mit `IntStream`:

```
final IntStream s = IntStream.of(5, 3, 6, 7, 1, 8, 2);  
final int max = s.max().orElse(-1);  
System.out.println(max);
```



- **Schlechte**, wesentlich aufwändigere Lösung mit `Stream`:

```
final Stream<Integer> s = Stream.of(5, 3, 6, 7, 1, 8, 2);  
s.sorted(Collections.reverseOrder())  
  .findFirst().ifPresent(System.out::println);
```



- Konvertierung (`int/Integer`), Sortieraufwand (!) etc.
- **Achtung**: Weil Stream-Lösungen sehr kompakt sind, kann man total «unauffällig» **sehr** ineffiziente Lösungen realisieren!

Exkurs: Auto[Un]Boxing

AutoBoxing und AutoUnboxing - Grundlagen

- Bereits in Java 5 (Einführung von Generics) wurde die (umstrittene) Technik des «auto boxing» und «auto unboxing» eingeführt.
- Zweck: Brückenschlag zwischen elementaren und Klassentypen.
 - `int` vs. `Integer`, `long` vs. `Long`, `float` vs. `Float` etc.
- Motivation: Generics sind «nur» mit Klassen nutzbar, aber nicht mit Elementartypen.
 - `List<int>` geht **nicht**, sondern nur `List<Integer>`.
- Vor Java 5 musste man darum immer **explizit** konvertieren:

```
final int intValue = 18;  
final Integer intObject = Integer.valueOf(intValue);  
final int value = intObject.intValue();
```

→ Schon wieder Boilerplate-Code, der unübersichtlich ist.

- Nebenbei: `new Integer(int)` ist seit Java 9 **deprecated**!

AutoBoxing und AutoUnboxing - Funktion

- Seit Java 5 kann man (z.B.) **Integer**-Objekte direkt **int**-Werten (und auch umgekehrt) zuweisen! Der Compiler fügt **automatisch** die dafür notwendige Konvertierung ein:

```
int intVal = 18;  
final Integer intObject = intVal;           // AutoBoxing!  
intVal = intObject;                         // AutoUnBoxing!
```

- Das funktioniert für sämtliche **elementaren** Datentypen.

- Darum ist seit Java 5 auch folgendes erlaubt und möglich:

```
final Stack<Integer> stack = new Stack<>();  
stack.push(18);                               // AutoBoxing!  
final int value = stack.pop();                 // AutoUnBoxing!
```

- Das ist in vielen Situationen sehr bequem! Das Feature wurde aber zu Beginn sehr kontrovers diskutiert, weil es auch Overhead produziert, wenn es nicht beabsichtigt ist.

AutoBoxing und AutoUnboxing – Bewertung und Einfluss

- Dieses Sprachfeature ist nicht unumstritten: Durch ungeschickt formulierten Code kann man viele unnötige Konvertierungen («teure» Objekterzeugung) provozieren!
 - Hat einen höheren Ressourcenverbrauch (Rechenzeit und Speicher) zur Folge, und belastet dann auch die Garbage Collection.
 - Bei Verarbeitung von grossen Datenmengen (Iterativ oder über Streams) kann sich der Aufwand natürlich sehr schnell Vervielfachen.
 - ➔ Typischer «Anfänger*innenfehler», weil nicht offensichtlich.
- Darum wurden in der Stream API zusätzliche Klassen, Interfaces und Methoden implementiert, welche einen effizienteren Umgang mit ausgewählten, elementaren Datentypen erlauben!
 - Beispiele: **IntStream**, **LongStream** and **DoubleStream**
 - ➔ Unbedingt verwenden, wenn elementare Typen ausreichen!

Wichtige Hinweise und Empfehlungen

Wichtige Hinweise



- Die Stream API ist sehr mächtig, man benötigt aber einige Grundlagen, um sie sicher und effizient nutzen zu können.
- «Schlechte» (ineffiziente) Lösungen sind im imperativen Stil meist deutlich einfacher identifizierbar, als in deklarativen Lösungen.
 - Aufgrund der Kompaktheit fallen bei Stream-Pipelines «teure» Umwege nicht so unmittelbar auf.
- Die Erstellung einer kurzen, prägnanten, effizienten Lösung mit der Stream API dauert (am Anfang) manchmal oft länger (!) als eine klassische Lösung.
 - Sammeln Sie Erfahrung, hinterfragen Sie dabei aber immer wieder kritisch Ihre Lösung auf deren leichte Verständlichkeit und testen Sie sie seriös!



Empfehlungen

- Beim aktuellen Kenntnisstand eher zurückhaltend einsetzen:
Nur dann, wenn Ihnen **vollständig klar** ist, wie es genau funktioniert!
- Beginnen Sie mit kleinen, einfachen Beispielen (z.B. IDE-Refactorings).
- Brechen Sie eine Pipeline im Zweifel besser in mehrere Schritte auf, anstatt die vermeintlich ultimative «Super-Pipeline» zu schreiben.
 - Einfachheit und Verständlichkeit.
- Filtern (reduzieren) Sie immer so früh wie möglich zur Reduktion der Datenmenge.
 - Die Reihenfolge der Operationen ist sehr entscheidend in Bezug auf den Ressourcenbedarf.
- Passen Sie auf im Umgang mit **elementaren** Datentypen.
 - Pipelines «verstecken» **Auto[Un]Boxing**-Aufrufe sehr gut.
 - Nutzen Sie ggf. die speziell zur Verfügung gestellten Streams!

Zusammenfassung

- Stream API zur Stream-Orientierten Verarbeitung von Daten.
 - Nicht verwechseln mit I/O-Streams.
- Eng gekoppelt mit der Verwendung von funktionaler Programmierung mit Lambdas.
- Häufig genutzt bei der Verarbeitung von Daten aus Collections.
- Basiert auf Supplier-, Intermediate- und Terminal-Operationen.
- Erlaubt sehr kompakte, deklarativ formulierte Pipelines.
 - Gefahr ineffizienter Lösung, immer kritisch hinterfragen.
- Technik des Auto-Boxing und -Unboxing von elementaren Datentypen.



Fragen?